Creating a dataset of Funda house listings and estimating time to sell a house

Una Garcia

12722707

2022-06-27

Supervisor
Robin Langerak

Second examiner
Roman Pankow

Bachelor thesis Information Science

Faculty of Science

University of Amsterdam

Abstract

Substantive data about houses with their features on the Dutch real estate market is not accessible to the general public. The most substantive API for this market is not an option for public, independent research, because access to this API requires profitability for the host, Brainbay. This thesis proposes an automatic method of scraping the desired information from sold Funda house listings and addresses the many considerations that have to be investigated when cleaning this data. After cleaning and pre-processing the data, the Python module TPOT was used to generate a pipeline model for estimating the amount of days to sell a house. The final model performed slightly better than all used baselines: mean, median and mode.

1 Introduction

Substantive data about houses with their features on the Dutch real estate market is not accessible to the general public. The largest Application Programming Interface (API) for the Dutch real estate market is owned by a company called BrainBay, which is a subsidiary of the ”Nederlandse Vereniging van Makelaars” (NVM, translated: Dutch Association for Real Estate). Brainbay states, however, that access to their API is only granted under certain conditions, the most important of which is that the use of the API grants the NVM added value (brainbay, n.d.). For the purpose of independent, public, research, this is not an option. Therefore, there is no publicly available data regarding the housing market and as a result the validity of research in this area is compromised. A potential workaround is to obtain the desired data through the publicly available website of Funda. Funda is another subsidiary of the NVM and it is the largest online platform for real estate transactions in the Netherlands (Steegmans & de Bruin, 2021). Brainbay manages the data for all the houses and other real estate that are listed on the Funda website. Funda provides access to data such as, amongst others, photo’s, address, (living) area, layout, isolation quality and heating type. The website also keeps sold real estate on the platform, for a limited time of approximately a year after sale.

With the data that is shown on a webpage of a house listing, it is possible to create a dataset containing sold houses and many features that are listed on their page. However, the added value of this data is limited as the resulting dataset suffers from incompleteness, inconsistency and inaccuracy. It is therefore not only important to create a dataset on details of houses that have been sold, it is also of the utmost importance to post-process this dataset. Creating such a dataset and cleaning it adequately supports the augmentation of scientific knowledge and makes it possible to apply machine learning models on the data. In addition, its societal relevance stems from the fact that the constructed dataset greatly enhances the transparency of the housing market for the general public. There is data to be found about costs of living (Centraal Bureau voor de Statistiek, 2022c) , of mean WOZ-values (Centraal Bureau voor de Statistiek, 2022a) and of house availability (Centraal Bureau voor de Statistiek, 2022b) however these datasets do not contain house features. An organisation that does offer substantive house (and other real estate) feature data, is Matrixian Group (Matrixian Group, n.d.), but this comes at a price, making the data inaccessible for some parts of society and also not usable for this thesis. Otherwise, no other publicly accessible data sources regarding the housing market could be found.

This thesis proposes an automatic method of scraping the desired information from Funda and addresses the many considerations that have to be investigated when cleaning this data. Although this is not the main contribution of this thesis, it is also demonstrated how the constructed dataset can be used to create a model estimating the time to sell a house. Thus, there are two main questions that this thesis focusses on. Firstly, how can a substantive dataset of houses be created, using Funda’s website as the data source? Secondly, how well can this dataset be used for predictive modelling purposes?

2 Background

Krause and Lipcomb (2016) wrote an article which is specifically concerned with working with inherently inconsistent data within the real estate sector. They proposed a *Data Preparation and Workflow Index* (DPWI) score, based on a list of analytical workflow questions, and assessed multiple studies that used real estate data and were published between the years 1980 and 2010. It was found that the data cleaning process was generally the worst documented part in studies. With this knowledge, this thesis attempts to provide a satisfactory documentation of the used cleaning process.

Abdi and Abolmakarem (2021) describe a rule-based algorithm for estimating time to sell a house in Iran. Although the findings presented in this paper are not easily generalized to the Dutch housing market, the house features used in this paper have been used in this thesis as an indication of what features should be included in a model that can be used to predict the time before a house is sold. However, fitting with the previous article, this article does not describe how the used data is retrieved and cleaned.

There is relatively little literature that specifically describes the process of finding, retrieving and cleaning house data, nor is there literature for other specific data that may be generalisable to the housing market. However, there are several guidebooks for processing and cleaning data in general, in this case with Python: one of these is written by Brownlee (2020), which focuses purely on the cleaning process and does not describe predictive models. This handbook was used in this thesis as an overview of the steps required in the data cleaning process. Another handbook is by Müller and Guido (2016), which places much less emphasis on cleaning and more on machine learning models and their implementation. Even though this thesis made use of tree-based automated machine learning for building a model (Le et al., 2020), and thus did not implement the algorithms manually, like shown in this handbook, the interpretation of the finally found predictive model has, in large part, been derived from this handbook. Like Müller and Guido, (2016), Van der Plas (2016) is a handbook that also describes several machine learning algorithms, however this work was used more as a guideline for creating appropriate visualisations of results with Python.

The article by Krause and Lipscomb (2016) (amongst others) explicitly states that there is a notable shortage of focus on the data cleaning and preparation process in existing literature and education, especially considering how important this process is for any further use of data. Brownlee (2020) agrees and states that the cleaning of data is an under-represented part of data processing, even though it is the most time-consuming and requires a certain amount of subjectivity and creativity. McKinney (2012), another comprehensive handbook on the entire data processing and modelling process, makes similar claims.

Regarding the raw data retrieval process, Glez-Peña et al. (2014) describe methods for web-scraping: programmatically extracting information from HTML that web-pages are built from. This article was used as an overview for existing web scraping utilities, as the raw data used for this thesis was retrieved by web-scraping Funda's website.

No literature was found that specifically addresses data about houses in the Netherlands, from Funda or other data sources.

3 Methodology

The process of this study consisted of four main parts: (1) obtaining raw HTML data, (2) extracting usable data from the HTML, (3) cleaning the extracted data and (4) building a predictive model estimating the amount of days to sell a house.
In all of these steps, the following libraries were used, *BeautifulSoup4* (version 4.10.0, Richardson, n.d.) , *requests-html* (version 0.10.0, Reitz, 2020/2022), *pandas* (version 1.3.5, Reback et al., 2021), *TPOT* (version 0.11.7, Weixuan Fu et al., 2020) and *dateparser* (1.1.1, Scrapinghub, n.d.). Several features were processed using regular expressions. All used patterns are listed in Appendix B. A repository providing access to all used scripts and data is available at https://github.com/raikenu159/Bsc_thesis_Funda.

3.1 Retrieving HTML data

To obtain the raw data, a snapshot of a part of Funda's website https://www.funda.nl/ was made by scraping it. Scraping started at the first result page of sold residential houses, ordered by sold date in ascending order. Conveniently, the options of filtering by only sold units, by residential houses and ordering the results are included in the website URL (e.g.: https://www.funda.nl/koop/heel-nederland/verkocht/woonhuis/sorteer-afmelddatum-op/p1/). By directly manipulating this URL the following modes can be enabled: excluding rental units, selecting only sold units, filtering only residential houses and sorting by ascending selling date. Because the website content changes continually and thus the amount of result pages is not constant throughout the web scraping process, the scraping process started at the final page and ended at the first. This way, the pages that are uncertain to exist towards the end of the scraping process, are retrieved early on. After retrieving all the result page URL's, they were iterated over and with each result page, the webpage URL of each individual listed house was available. Using the Python module *requests-html* (Reitz, 2020/2022) to also load the JavaScript-generated content on a listing page when sending a request, the HTML content of all listing pages was retrieved and saved in a text format. The process started on 19 April 2022, and finished 5 days later, on 24 April. This resulted in a text file containing 34.7 GB of data from n = 127717 house listing web pages.

3.2 Extracting and cleaning usable data from HTML

To extract desired information from the retrieved raw HTML, the Python library *BeautifulSoup4* (Richardson, n.d.) was used. In order to determine a preliminary feature selection, a subsample of 5000 houses was processed, In this subset of 5000 houses, 56 house features were found. After the features had been extracted, cleaning the dataset was the next step. This consisted of several steps, which are described in the following sections.

3.2.1 Filtering features

Before cleaning the house features, a selection of features and rows was made. This was done to limit processing of features that would ultimately not be used and to remove features that too few instances actually contain information about. First, only house entries were selected of which the feature “Status” was “Verkocht” (sold). This was because some house entries, likely due to differing layout, were retrieved incorrectly and had their features shifted in columns and so their values did not represent the feature they fell under. Next, columns where 30% or more of the rows had a missing value were dropped. 25% was considered, but this removed energy label and storage type, which were desired enough to keep, especially because these features are categorical. This means that missing values can be represented as a separate feature by one-hot encoding, which is described at a later step. After filtering columns, all rows where 25% or more of the remaining columns contained missing values were also dropped. These filters were specifically chosen to not be very strict, so that remaining missing values can be dealt with later on, either by imputing values or removing more rows.

After this, 7 self-selected features were also removed, for different reasons: (1) “Laatste Vraagprijs” (last sold price) and (2) “Oppervlakte” (area) were removed because they were duplicates of other retrieved features. (3) “Eigendomssituatie” (ownership) was removed because it was not informative enough, due to indescriptive ‘zie akte’ (see document) values. (4) “Looptijd” (duration listed) was removed because it was calculated separately, using the features *date_listed* and *date_sold* after they were converted to date-time formats. (5) “Status” was removed because it was only used to select well-retrieved houses. (6) “Cv-ketel” was removed because it contained unnecessary extra information. The useful information it contained was already contained in the features *type_heating* and *type_warm_water*. Finally, (7) “Achtertuin” was removed because it contained inconsistent formatting.

After this process, 29 features and 116403 rows remained. An overview of the features and example non-missing values are shown in Table 1.

Table 1: Overview of 29 remaining features after filtering and renaming features. Shows feature name and example non-NaN value

Feature	Example non-NaN value
url	https://www.funda.nl/koop/verkocht/veenendaal/huis-88077563-gerard-terborchstraat-17-a/
street	Gerard Terborchstraat 17 a
postal_city	3904 TE Veenendaal Franse Gat
sale_broker	Zonnenberg Makelaardij B.V.
date_listed	26 februari 2022
date_sale	18 maart 2022
sold_price	\xe2\x82\xac 365.000 k.k.
year_build	1984
area_living	127 m\xc2\xb2
area_plot	181 m\xc2\xb2
volume	453 m\xc2\xb3
n_photos	39
n_rooms	6 kamers (5 slaapkamers)
n_sleepingrooms	5.0
n_floors	2 woonlagen en een zolder
type_house	Eengezinswoning, eindwoning
type_build	Bestaande bouw
type_facilities	Elektra
type_isolation	Dakisolatie, dubbel glas en vloerisolatie
type_heating	Cv-ketel, open haard en gedeeltelijke vloerverwarming
type_warm_water	Cv-ketel
type_yard	Achtertuin en voortuin
type_orientation_yard	Gelegen op het oosten bereikbaar via achterom
type_parking	Op eigen terrein en openbaar parkeren
type_roof	Zadeldak bedekt met pannen
type_energy_label	C Wat betekent dit?
type_locality	In woonwijk
type_storage	Vrijstaande houten berging
type_bathroom_facilities	Ligbad, douche en toilet

3.2.2 Cleaning remaining features

Only the features that were not planned to be used for fitting and/or describing data did not have to be cleaned. These are *url*, *street* and *sale_broker*. All other features required preprocessing.

Extract place

City or place was extracted from the URL, instead of the *postal_city* feature, as this also contained distinctions between different places with the same name, such as "*alphen-ge*" and "*alphen-nb"*, respectively referring to Alphen in the province of Gelderland and another Alphen in the province of Noord-Brabant. Postal code was also extracted using a regular expression, however this feature was not used in the end.

Extract digits from to-be numerical features

sold_price, area_living, area_plot, volume, n_photos, n_rooms and *n_floors* are features that contain a single desired number value, but also contain non-numerical data. Regular expressions were used to extract the digits from these string values and the extracted values were converted to float data types, for example with the feature *sold_price*, where an unprocessed value of "\xe2\x82\xac 1.850.000 k.k." is converted to "1850000.0"

Parse dates

date_listed and *date_sale* are features that contain date-time data and have to be parsed as such. For this, the Python library *dateparser* was used, as this can parse dates that are written out in several languages, including Dutch. For example, a date_sale string value of "28 maart 2022" should be converted to a date-time value "2022-03-28". After these two features are parsed and converted to date-time format, the target feature for modelling was calculated by taking the difference between *date_listed* and *date_sale* in days, Values less than 0 were dropped, with the assumption that these were placed on Funda after they were already sold and thus were not useful for the modelling purpose. Build year was simply converted to an integer value.

Split type features

All features with the prefix "place_" are categorical features which are not usable as is. Different features are described in a single piece of string, separated by comma's and/or the word "en" (and), as shown in Table 1. For all *type_*- features, except for *type_energy_label, type_orientation_yard* and *type_place, t*his was solved in four steps: Firstly, all these features were converted to lowercase. Secondly, the spaces were replaced with underscores. Thirdly, the values were split into lists with each element split by either a comma followed by an underscore (",_") or an "en" surrounded by underscores ("_en_"). Finally, for the purpose of one-hot encoding in a later step, the resulting lists were converted back to a string, where the values were split by a vertical bar ("|"). The result of this cleaning process is showcased in Table 2 below. *type_orientation_yard* and *type_energy_label* are not included, because these are not yet processed at all in this step.

Table 2: Type features split by vertical bars ("|") after processing. Bold features still require further processing before encoding.

Feature	Example value before split	Example value after split
type_place	veenendaal	veenendaal
type_house	Eengezinswoning, eindwoning	eengezinswoning|eindwoning
type_build	Bestaande bouw	bestaande_bouw
type_facilities	Elektra	elektra
type_isolation	Dakisolatie, dubbel glas en vloerisolatie	dakisolatie|dubbel_glas|vloerisolatie
type_heating	Cv-ketel, open haard en gedeeltelijke vloerverwarming	cv-ketel|open_haard|gedeeltelijke_vloerverwarming
type_warm_water	Cv-ketel	cv-ketel
type_yard	Achtertuin en voortuin	achtertuin|voortuin
type_parking	Op eigen terrein en openbaar parkeren	op_eigen_terrein| openbaar_parkeren
type_roof	Zadeldak bedekt met pannen	zadeldak_bedekt_met_pannen
type_locality	In woonwijk	in_woonwijk
type_storage	Vrijstaande houten berging	vrijstaande_houten_berging
type_bathroom_facilities	Ligbad, douche en toilet	ligbad|douche|toilet

At this point, not all of the categorical features are yet ready to be encoded. *Energy label, yard orientation, house type*, *bathroom facilities (*for numbers in values, these are not shown in the example*)* and *roof types* still need to be processed further.

Energy label

For the energy label, only one of the letters A, B, C, D, E, F, G was extracted per house, using the ‘clean energy label” pattern, listed in Appendix B. Exceptionally high energy labels with plusses were regarded as only the letter, excluding the plus(ses). The reasoning for this is mentioned in the discussion. Any values other than one of the letters, such as “Niet beschikbaar” (unavailable), or “Niet verplicht” (not required), were regarded as ‘unspecified’.

Yard orientation and backside access

The information to be extracted from the feature *yard_orientation* is the described cardinal direction and whether it is described that the yard is accessible from the back. For the cardinal directions, one of eight cardinal directions (including diagonals) was extracted using regular expressions. For backside access, a new binary feature was added, describing the presence of described backside access.

Bathroom facilities

For the feature *bathroom_facilities,* sometimes a certain number of toilets is specified. However, because most of the entries don't have more than two toilets in a bathroom and because it is vague whether the amount of toilets of possibly multiple bathrooms are counted in *bathroom_facilities*, the numbers mentioned in *bathroom_facilities* were ignored. Only the presence of one or more of a feature was kept. For example, the value "ligbad|douche|2_toiletten" is converted to "ligbad|douche|toilet". In a later step, these, together with all other categorical features, are encoded to binary values.

House type and roof type

Some features contain values that are combinations of multiple features, also after the previous splitting process. This causes many more unique values in the features than there actually are, as showcased in Appendix C This is the case with the features *type_house* and *type_roof.* An example of a value which contains combined features is "eengezinswoning|tussenwoning_(dijkwoning)": 'dijkwoning' is desired as a separate feature, unconnected to 'tussenwoning'. Regular expressions were also used to extract all unique feature names present in the data and include them in the string values, split by vertical bars. Examples of pre- and post-processed *type_house* values are shown in Table 3.

Table 3: Examples of processed house type values, compared to their unprocessed counterparts.

	type_house before	type_house after
Example value 1	eengezinswoning|tussenwoning_(dijkwoning)	eengezinswoning|tussenwoning|dijkwoning
Example value 2	bungalow|vrijstaande_woning_(semi-bungalow)	bungalow|vrijstaande_woning|semi-bungalow

type_roof contains a similar problem to *type_house*. An example value is "dwarskap_bedekt_met_asbest". The desired features from this string are "dwarskap" and "asbest". Many values in this feature are like this. This was solved in the same way as with the house types; the values that are desired to be detected as separate features were extracted with a regular expression. Examples of the roof type processing result are shown in Table 4.

Table 4: Examples of processed roof type values, compared to their unprocessed counterparts.

	type_roof before	type_roof after
Example value 1	zadeldak_bedekt_met_bitumineuze_dak bedekking|pannen	zadeldak| bitumineuze_dakbedekking| pannen
Example value 2	zadeldak_bedekt_met_pannen	zadeldak|pannen

Imputing missing values in numerical features

Because many machine learning algorithms do not work on data with missing (NaN) values, a decision has to be made on what to do with them. For the categorical features, a feature that was not specified was simply listed as a separate, unique value, to be encoded as an extra feature later on. For numerical features, however, a different approach is required. In this case, for some features, rows missing a value for the feature are simply removed, because these features are considered as basic, essential features that all houses should contain. Features that were treated this way are *sold_price, year_build, area_living, area_plot* and *n_floors.* Next, for the numerical features which were considered slightly less essential, the missing values of remaining numerical features were replaced with the median of the corresponding feature. Features where this was performed are: *volume, n_photos, n_rooms and n_sleepingrooms.* This process removed 7103 rows.

Removing outliers and rows with infrequent places

Before the final step of encoding all categorical features, outliers of numerical features and rows with places that occurred less than 10 times were removed. This is because they can greatly worsen the performance of machine learning algorithms. For the numerical features *sold_price year_build area_living area_plot volume, n_photos, n_rooms, n_sleepingrooms,* and *n_floors,* outliers were considered as values which resided more than 3 standard deviations away from the mean and these were subsequently removed. This resulted in a total of 13 346 rows being removed.

Encoding (multi-label) categorical features.

To make the categorical features usable for machine learning algorithms, they have to be represented in a numerical form. Because the features do not represent an order of values, such that one value can be mathematically larger than another, simply encoding each unique to a unique number is not enough. Instead, each unique value in a categorical feature is assigned its own column, where the presence of the feature is represented as a binary value with 0 meaning not present and 1 meaning present. This is called one hot encoding (Brownlee, 2020). In this thesis, there are two slightly different cases where encoding has to be performed: in the first case, features contain only a single value which needs to be encoded. In this case, scikit-learn's preprocessing.OneHotEncoder function was used to encode these single-valued categorical features, after they were converted to a categorical data type. In the second case, each feature consists of multiple values which have to be binarised. In this case, scikit-learn's preprocessing.MultiLabelBinarizer function was used, after converting the values to sets, as

this is required for the function to perform. An example of the feature type_bathroom_facilities being encoded is shown in Table 5 below.

Table 5: Showcase of multi-label one hot encoding. The first row describes the non-encoded type_bathroom_facilities value and the rest of the rows describe the encoded result.

Feature	Example value 1	Example value 2
type_bathroom_facilities before encoding		ligbad|toilet
type_bathroom_facilities_	1	0
type_bathroom_facilities_douche	0	0
type_bathroom_facilities_jacuzzi	0	0
type_bathroom_facilities_ligbad	0	1
type_bathroom_facilities_sauna	0	0
type_bathroom_facilities_stoomcabine	0	0
type_bathroom_facilities_toilet	0	1
type_bathroom_facilities_zitbad	0	0

3.3 Generating a predictive model

After the data was cleaned and preprocessed, the Python library TPOT (Weixuan Fu et al., 2020) was used to generate a model for estimating time to sell a house, based on the supplied features. This library is an automated machine learning tool which tests and optimizes several different machine learning pipelines using genetic programming. When the searching process is finished or terminated early, the best performing pipeline is returned. This way, it is also possible to see what functions were used on the data.

Using sklearn's *train_test_split* function, the dataset was split into a training set and feature set, with the training set being 75% of the data (71580 instances), as this is an often used ratio and is also the default value used by TPOT. The option *shuffle* was used and the *random_state* was 42. TPOT's TPOTRegressor class was used to search pipelines over 20 generations of population size 100 and a random_state of 42 N_jobs was 3 and 32 GB of RAM was available. Because the used machine for searching a model operated on Linux, the Python library *multiprocessing* was imported and the *multiprocessing.start_method* was set to "forkserver". This was required for using multiple cores during the searching process and is a known issue described in the scikit-learn FAQ (scikit-learn developers, n.d.). With these settings, the searching time was approximately 32 hours.

4 Results

4.1 Dataset description

The resulting complete dataset, after cleaning and preprocessing, consisted of n = 95441 instances of sold houses and 1346 features. The houses in the dataset were sold from 14 October 2019 to 15 April 2022. The target feature *days_to_sell* ranges from 0 to 279 days, with a mean value μ = 27.8, standard deviation σ = 26.2 and a median of 21. A box plot describing the value distribution of all numerical features, including *days_to_sell* is shown in Figure 1. For visualising purposes, features have been scaled between 0 and 1, so their respective minimum and maximum values are displayed in Table 6.

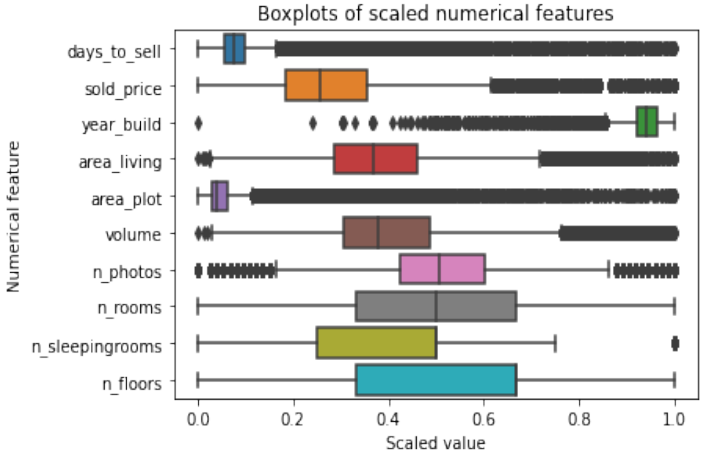

Figure 1: Box plots displaying distributions of all numerical features, with values scaled between 0 and 1.

Table 6: Minimum and maximum values of all numerical features

Feature name	Minimum value	Maximum value
days_to_sell	0.0	279.0
sold_price	78500.0	1175000.0
year_build	1200.0	2024.0
area_living	39.0	259.0
area_plot	12.0	4478.0
volume	102.0	968.0
n_photos	0.0	73.0
n_rooms	2.0	8.0
n_sleepingrooms	2.0	6.0
n_floors	1.0	4.0

Most houses were sold in Almere, followed by Tilburg, Eindhoven, Utrecht and more. The top 20 places of sold houses are displayed in Figure 2. This ranking does not show a representative distribution of the feature however. This is displayed in a kernel density estimator plot in Figure 3, which shows that by far the most places represent less than 0.25% of all sold houses.

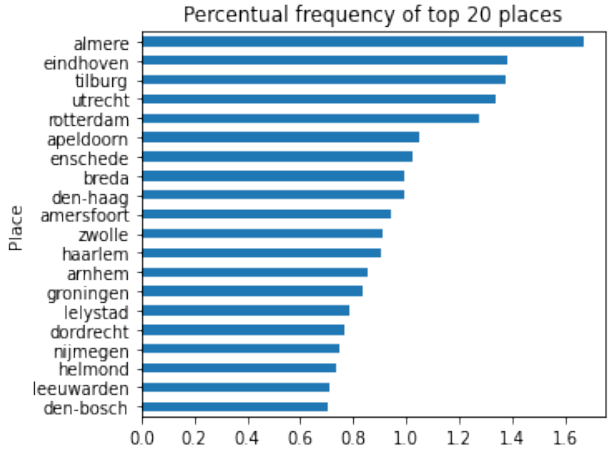

Figure 2: Horizontal bar plot displaying the top 20 places where most houses were sold in the Netherlands.

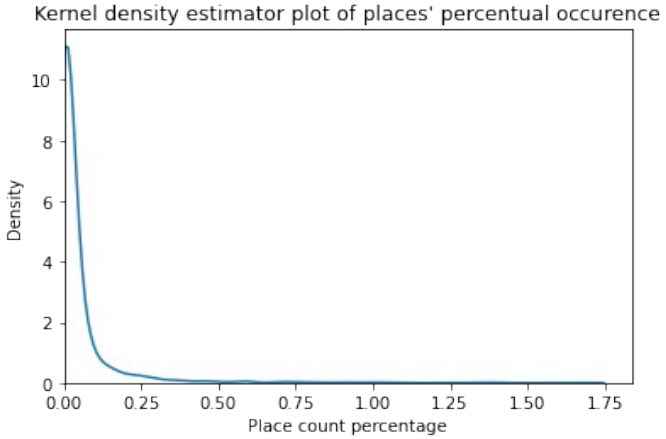

Figure 3: Kernel density estimator plot displaying the distribution of place counts. It shows a long right-tail, showing the large amount of places where close to 10 houses were sold.

Type features and amount of new features

As each *type_*- feature was one-hot encoded, each feature split into a number of separate features, corresponding to the amount of unique values that were present in the original feature. The amount of unique values found for each *type_*- feature, scaled logarithmically, are displayed in Figure 4. The feature *place* contained by far the most amount of unique values, also after removing values which occur less than 10 times.

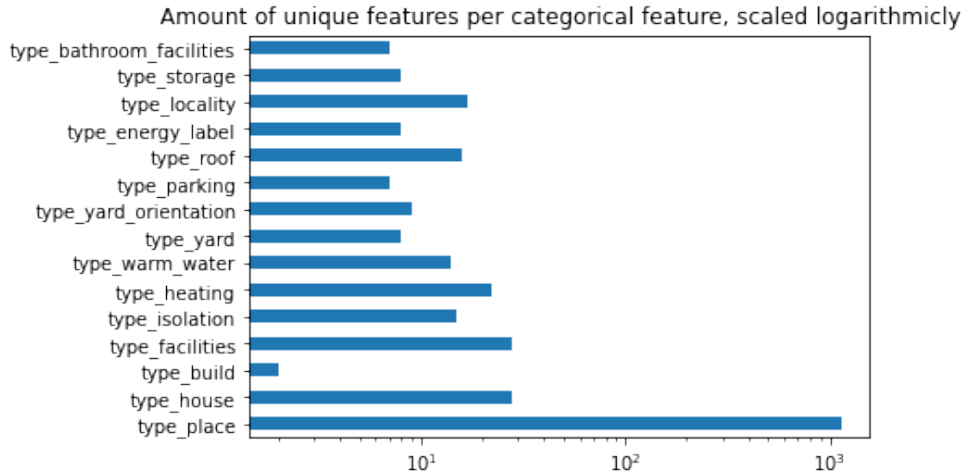

Figure 4: Unique feature counts of categorical features after encoding unique values, scaled logarithmically.

4.2 Predictive model

TPOT's regressor class considers twelve different regression-based predictive algorithms, excluding preprocessors and feature selectors. Included algorithms are ensemble methods, such as gradient boost and ADA(ptive) boost, K-neighbors regression, XGBoost regression and more (*EpistasisLab/Tpot*, 2015/2022). Using the root mean square error (RMSE) as a scoring function, the best performing model on the test set is a two-step pipeline containing a Gradient Boosting Regressor (GBR) and a cross-validated Lasso Least Angle Regression (Lasso Lars-CV). These input features for this pipeline consisted of 1345 features, however, as part of the pipeline, an extra feature was added by the Lasso Lars-CV, consisting of its own predicted values. Thus, the GBR was performed on 1346 features, most of which consisted of encoded place values. Options of the GBR included 100 estimators, a least-squared loss function, a learning rate of 0.1 and a maximum depth of 10. The full pipeline with all options is shown in Appendix D. This pipeline, together with all other tested pipelines during the searching process, was trained on a training set size of 75%, based on all cleaned and pre-processed features Thus, the training set consisted of 71580 instances and 1345 features.

The found model had an RMSE of 23.444, compared to a mean baseline RMSE of 25.451, a median baseline RMSE of 26.292 and a mode baseline RMSE of 26.561. This is also displayed in Figure 5. The found model performed slightly better on average than all used baselines.

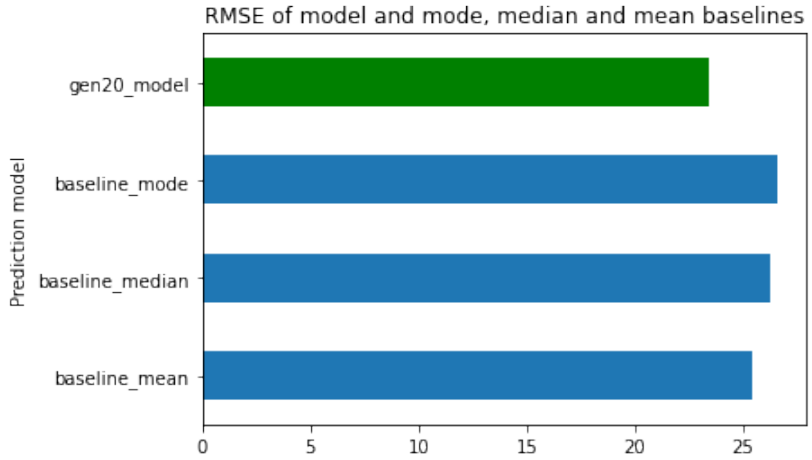

Figure 5: Bar chart displaying RMSE values of generated model and mode, median and mean baselines. The model outperformed all baselines.

The maximum error of the model was also slightly less on the test set, compared to all used baselines: the model maximum error was 244.117, compared to a mean baseline maximum error of 250.123, a median baseline maximum error of 257.0 and a mode baseline maximum error of 258.0, also displayed in Figure 6.

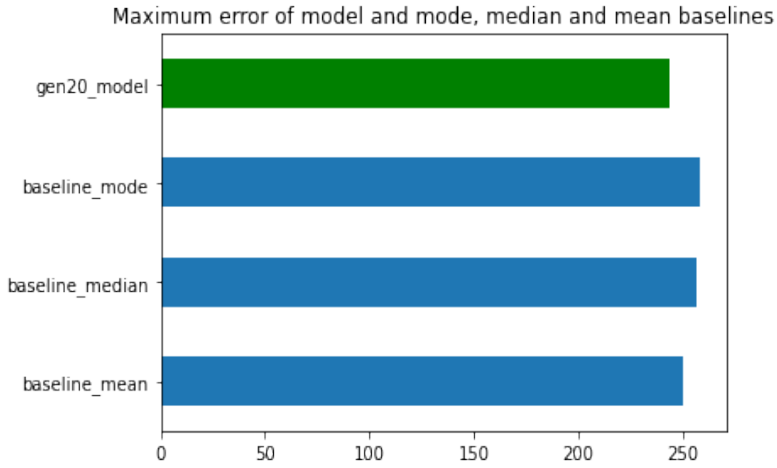

Figure 6: Bar chart displaying maximum error values of generated model and mode, median and mean baselines. The model outperformed all baselines.

5 Conclusion

The first main question asked in this thesis was “how can a substantive dataset of houses be created, using Funda’s website as the data source?”. The answer to this question is a four-step process. Firstly, obtaining raw HTML data from sold houses on Funda. Secondly, extracting useful information raw HTML. Thirdly, the largest step, cleaning the parsed data, before the final step of creating a predictive model.
This thesis focused mostly on the cleaning process, as this is an often an under-documented part of research publications, even though it is an essential part of making data usable for research or other applications. Besides, because real-world data is often inherently inconsistent, the cleaning process is subjective to a certain degree, which adds to the importance of its documentation. Cleaning and preprocessing steps in this thesis ranged from simply renaming and reordering features and changing data-types, to extracting and encoding meaningful string data and handling missing values and outliers. Especially the later steps were generally more subjective.

Using the created dataset, the second main question of this thesis could also be answered. The question was “how well can this dataset be used for predictive modelling purposes?” To create a predictive model, *TPOT* was used as an automated predictive model searching tool. The model found with this method performed slightly better, on average, compared to all used baselines: mean, median and mode. Both in measures of RMSE and maximum error. This indicates that the house data retrieved from Funda’s website and cleaned as described in this thesis, can be used for creating models for estimating time to sell a house, that at least perform better than simply always guessing either the mean, median or mode selling time.

6 Discussion

Data source

The dataset retrieved and used in this thesis only consisted of data of around one year worth of sold houses in the Netherlands, through the platform Funda, during a heated housing market. This leads to a bias in the data and thus it is not necessarily representative of the general housing market in the Netherlands. The largest cause of this bias in this study is that the chosen data source, Funda, removes older houses from their website. Furthermore, not all house listings on Funda are consistent, and the absence of a feature on a listing does not necessarily mean that the house does not contain this feature. For example, there are listings which do not explicitly specify what type of method is used to facilitate warm water. This does not necessarily mean, however, that there is no warm water available.

Webpage HTML retrieval order

Because of the chosen ascending sale date order, some older listings which are in the first pages of the website (and thus scraped last) will not be retrieved, because these pages are removed during the scraping process. Likewise, some newer pages will be duplicated because they shift into lower page numbers when pages at the beginning lose entries. Images and neighbouring facilities were not considered at all, nor retrieved, as these are outside of the scope of this study. However, these features could still be considered in further research.

Exceptional energy labels

Energy labels of A+ and higher were not processed as such, but as only the letter. This was simply because, at the time of making the function for processing the energy label, a regular expression pattern that also encompassed plusses could not be thought of at the time. Thus, this was not entirely a wilfully chosen decision, as there is a significant difference between, for example, an A energy rating and an A++++ energy rating: an A rating implies energy consumption between 105 and 160 kWh/$m^{2.}$, and an A++++ rating implies no (0) net energy consumption at all (Funda, n.d.).

Limited searching time

The default amount of generations used by TPOT regression (and classification) classes is 100. As this thesis ended the searching process after 20 generations due to time constraints (this searching process took one and a half days), the amount of tested pipelines and parameters was also limited. Thus, letting the searching process continue for a longer time would likely have lead to a (slightly) better performing final model.

7 References

Abdi, F., & Abolmakarem, S. (2021). Mining a Set of Rules for Determining the Waiting Time for Selling Residential Units. *Journal of System Management*, *7*(2), 171–203. https://doi.org/10.30495/jsm.2021.1931499.1480

brainbay. (n.d.). *Brainbay-API - Realtime inzichten vastgoedmarkt*. Retrieved 10 June 2022, from https://brainbay.nl/producten/brainbay-api/

Brownlee, J. (2020). *Data preparation for machine learning*.

Centraal Bureau voor de Statistiek. (2022a). *Gemiddelde WOZ-waarde van woningen op 1 januari; eigendom, regio*. https://data.overheid.nl/dataset/20481-gemiddelde-woz-waarde-van-woningen--1-januari--regio#panel-resources

Centraal Bureau voor de Statistiek. (2022b). *Voorraad woningen; gemiddeld oppervlak; woningtype, bouwjaarklasse, regio*. https://data.overheid.nl/dataset/441-voorraad-woningen--gemiddeld-oppervlak--woningtype--bouwjaarklasse--regio#panel-description

Centraal Bureau voor de Statistiek. (2022c). *Woonlasten huishoudens; kenmerken huishouden, woning*. https://data.overheid.nl/dataset/450-woonlasten-huishoudens--kenmerken-huishouden--woning

EpistasisLab/tpot. (2022). [Python]. Epistasis Lab at Cedars Sinai. https://github.com/EpistasisLab/tpot/blob/435dc100ee9ee40b9e5a44f70e62ed1a659c86cd/tpot/config/regressor.py (Original work published 2015)

Funda. (n.d.). *Energielabel huis: Wat betekenen de letters A tot en met G*. Funda. Retrieved 23 June 2022, from https://www.funda.nl/meer-weten/energielabel/energielabel-huis-wat-betekenen-de-letters-a-tot-en-met-g/

Glez-Peña, D., Lourenço, A., López-Fernández, H., Reboiro-Jato, M., & Fdez-Riverola, F. (2014). Web scraping technologies in an API world. *Briefings in Bioinformatics*, *15*(5), 788–797. https://doi.org/10.1093/bib/bbt026

Krause, A., & Lipscomb, C. A. (2016). The data preparation process in real estate: Guidance and review. *Journal of Real Estate Practice and Education*, *19*(1), 15–42.

Le, T. T., Fu, W., & Moore, J. H. (2020). Scaling tree-based automated machine learning to biomedical big data with a feature set selector. *Bioinformatics*, *36*(1), 250–256.

Matrixian Group. (n.d.). *Real Estate Data Netherlands (7.8 million properties) | Datarade*. https://datarade.ai/data-products/real-estate-data-matrixian-group

McKinney, W. (2012). *Python for data analysis: Data wrangling with Pandas, NumPy, and IPython*. O'Reilly Media, Inc.

Müller, A. C., & Guido, S. (2016). *Introduction to machine learning with Python: A guide for data scientists*. O'Reilly Media, Inc.

Reback, J., Jbrockmendel, McKinney, W., Van Den Bossche, J., Augspurger, T., Cloud, P., Hawkins, S., Roeschke, M., Gfyoung, Sinhrks, Klein, A., Terji Petersen, Hoefler, P., Tratner, J., She, C., Ayd, W., Naveh, S., Garcia, M., JHM Darbyshire, … Skipper Seabold. (2021). *pandas-dev/pandas: Pandas 1.3.5* (v1.3.5) [Computer software]. Zenodo. https://doi.org/10.5281/ZENODO.5774815

Reitz, K. (2022). *requests-html: HTML Parsing for Humans.* (0.10.0) [Python]. https://github.com/kennethreitz/requests-html (Original work published 2020)

Richardson, L. (n.d.). *beautifulsoup4: Screen-scraping library* (4.11.1) [Python]. Retrieved 10 June 2022, from https://www.crummy.com/software/BeautifulSoup/bs4/

scikit-learn developers. (n.d.). *Frequently Asked Questions*. Scikit-Learn. Retrieved 14 June 2022, from https://scikit-learn/stable/faq.html

Scrapinghub. (n.d.). *dateparser: Date parsing library designed to parse dates from HTML pages* (1.1.1) [Python]. Retrieved 10 June 2022, from https://github.com/scrapinghub/dateparser (Original work published 2014)

Steegmans, J., & de Bruin, J. (2021). Online housing search: A gravity model approach. *Plos One, 16*(3), e0247712. https://doi.org/10.1371/journal.pone.0247712

VanderPlas, J. (2016). *Python data science handbook: Essential tools for working with data*. GitHub. https://github.com/jakevdp/PythonDataScienceHandbook

Weixuan Fu, Olson, R., Nathan, Grishma Jena, PGijsbers, Augspurger, T., Romano, J., PRONOjit Saha, Sahil Shah, Raschka, S., Sohnam, DanKoretsky, Kadarakos, Jaimecclin, Bartdp1, Bradway, G., Ortiz, J., Jorijn Jacko Smit, Jan-Hendrik Menke, … Carnevale, R. (2020). *EpistasisLab/tpot: V0.11.5* (v0.11.5) [Computer software]. Zenodo. https://doi.org/10.5281/ZENODO.3872281

Appendix

A. Images of house webpages

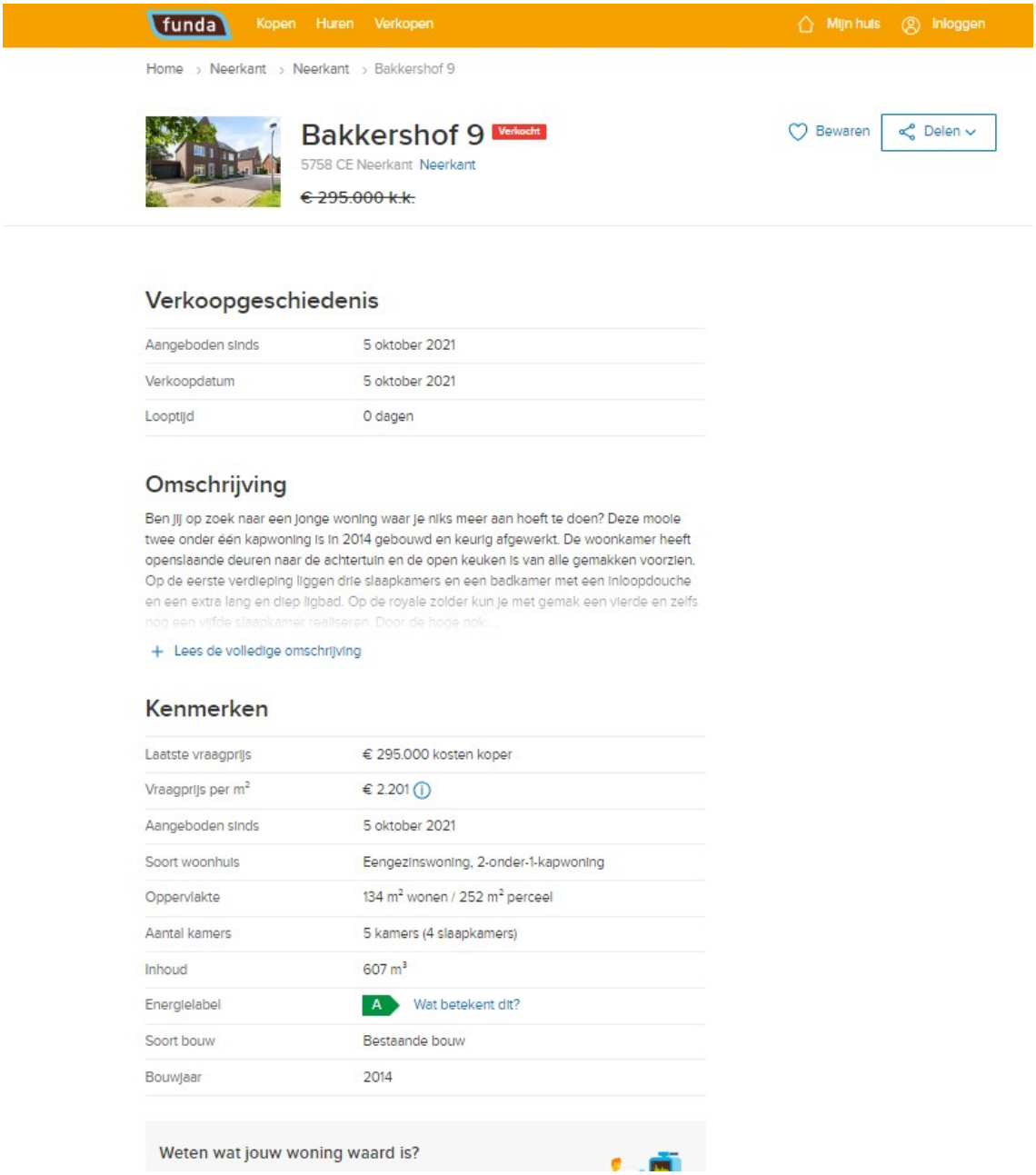

Figure 7: Listing page with little information and different layout

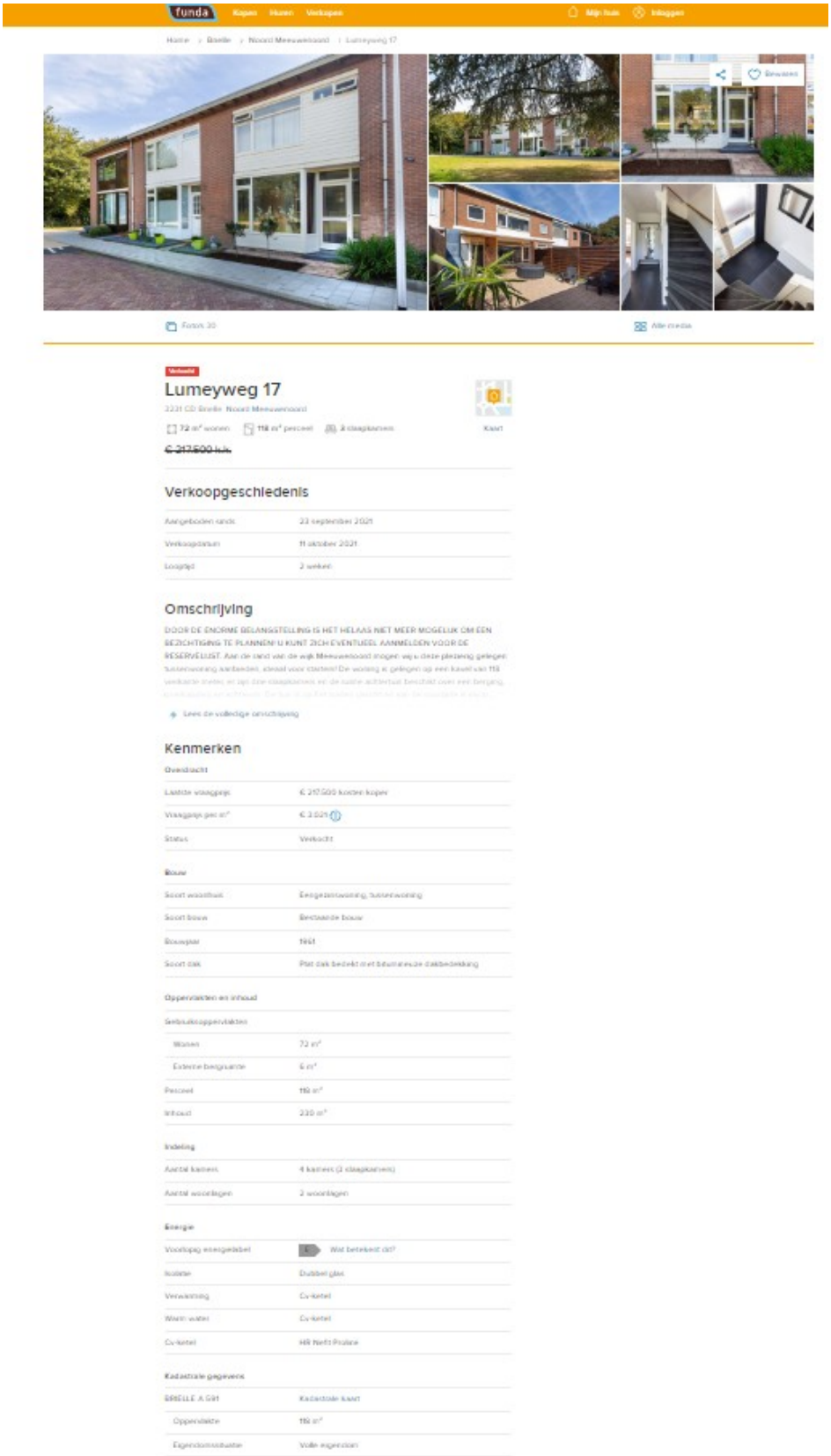

Figure 8: "Normal" house listing, containing images, living area, plot area and number of sleeping rooms early on.

B. Regular expression patterns

Extract digits from numerical features: "(ac)?(\d+) [mkvwa][^e]"

Clean energy label: "([ABCDEFG])"

Get yard orientation: "noorden|oosten|zuiden|westen|noordoosten|zuidoosten|zuid-westen|noordwesten"

Get yard backside access: "bereikbaar"

Clean housetype: "(2-onder-1-kapwoning|bungalow|eengezinswoning|eindwoning|geschakelde_woning|grachtenpand|halfvrijstaande_woning|herenhuis|hoekwoning|landgoed|landhuis|stacaravan|tussenwoning|verspringend|villa|vrijstaande_woning|woonboerderij|woonboot|woonwagen|bedrijfs-_of_dienstwoning|dijkwoning|drive-in_woning|hofjeswoning|kwadrant_woning|patiowoning|semi-bungalow|split-level_woning|waterwoning|wind-_of_watermolen|paalwoning)"

Clean bathroom facilities:

digit pattern = "\d+_"

facility pattern = "douche|toilet|jacuzzi|ligbad|zitbad|stoomcabine|sauna"

Clean roof types: "bitumineuze_dakbedekking|lessenaardak|mansarde_dak|plat_dak|dwarskap|schilddak|tentdak|zadeldak|samengesteld_dak|asbest|kunststof|kunststof|leisteen|metaal|pannen|riet|overig"

C. Sets of unique values

I. Unprocessed unique house type values

{'2-onder-1-kapwoning_(bedrijfs-_of_dienst-woning)',
'2-onder-1-kapwoning_(dijkwoning)',
'2-onder-1-kapwoning_(drive-in_woning)',
'2-onder-1-kapwoning_(hofjeswoning)',
'2-onder-1-kapwoning_(kwadrant_woning)',
'2-onder-1-kapwoning_(paalwoning)',
'2-onder-1-kapwoning_(patiowoning)',
'2-onder-1-kapwoning_(semi-bungalow)',
'2-onder-1-kapwoning_(split-level_woning)',
'2-onder-1-kapwoning_(waterwoning)',
'bungalow',
'eengezinswoning',
'eindwoning_(bedrijfs-_of_dienstwoning)',
'eindwoning_(dijkwoning)',
'eindwoning_(drive-in_woning)',
'eindwoning_(hofjeswoning)',
'eindwoning_(kwadrant_woning)',
'eindwoning_(patiowoning)',
'eindwoning_(semi-bungalow)',
'eindwoning_(split-level_woning)',
'eindwoning_(waterwoning)',
'geschakelde_2-onder-1-kapwoning_(bedrijfs-_of_dienstwoning)',
'geschakelde_2-onder-1-kapwoning_(dijkwon-ing)',
'geschakelde_2-onder-1-kapwoning_(drive-in_woning)',
'geschakelde_2-onder-1-kapwoning_(hofjeswon-ing)',
'geschakelde_2-onder-1-kapwoning_(kwadrant_woning)',
'geschakelde_2-onder-1-kapwoning_(patiowon-ing)',
'geschakelde_2-onder-1-kapwoning_(semi-bun-galow)',
'geschakelde_2-onder-1-kapwoning_(split-level_woning)',
'geschakelde_2-onder-1-kapwoning_(waterwon-ing)',
'geschakelde_woning_(bedrijfs-_of_dienst-woning)',
'geschakelde_woning_(dijkwoning)',
'geschakelde_woning_(drive-in_woning)',
'geschakelde_woning_(hofjeswoning)',
'geschakelde_woning_(kwadrant_woning)',
'geschakelde_woning_(patiowoning)',
'geschakelde_woning_(semi-bungalow)',
'geschakelde_woning_(split-level_woning)',
'geschakelde_woning_(waterwoning)',
'grachtenpand',
'halfvrijstaande_woning_(bedrijfs-_of_dienst-woning)',
'halfvrijstaande_woning_(dijkwoning)',
'halfvrijstaande_woning_(drive-in_woning)',
'halfvrijstaande_woning_(hofjeswoning)',
'halfvrijstaande_woning_(kwadrant_woning)',
'halfvrijstaande_woning_(patiowoning)',
'halfvrijstaande_woning_(semi-bungalow)',
'halfvrijstaande_woning_(split-level_woning)',
'halfvrijstaande_woning_(waterwoning)',
'herenhuis',
'hoekwoning_(bedrijfs-_of_dienstwoning)',
'hoekwoning_(dijkwoning)',
'hoekwoning_(drive-in_woning)',
'hoekwoning_(hofjeswoning)',
'hoekwoning_(kwadrant_woning)',
'hoekwoning_(patiowoning)',
'hoekwoning_(semi-bungalow)',
'hoekwoning_(split-level_woning)',
'hoekwoning_(waterwoning)',
'landhuis',
'tussenwoning_(bedrijfs-_of_dienstwoning)',
'tussenwoning_(dijkwoning)',
'tussenwoning_(drive-in_woning)',
'tussenwoning_(hofjeswoning)',
'tussenwoning_(kwadrant_woning)',
'tussenwoning_(paalwoning)',
'tussenwoning_(patiowoning)',
'tussenwoning_(semi-bungalow)',
'tussenwoning_(split-level_woning)',
'tussenwoning_(waterwoning)',
'verspringend_(bedrijfs-_of_dienstwoning)',
'verspringend_(drive-in_woning)',

'verspringend_(kwadrant_woning)',
'verspringend_(patiowoning)',
'verspringend_(semi-bungalow)',
'verspringend_(split-level_woning)',
'villa',
'vrijstaande_woning_(bedrijfs-_of_dienst-woning)',
'vrijstaande_woning_(dijkwoning)',
'vrijstaande_woning_(drive-in_woning)',
'vrijstaande_woning_(hofjeswoning)',
'vrijstaande_woning_(patiowoning)',
'vrijstaande_woning_(semi-bungalow)',
'vrijstaande_woning_(split-level_woning)',
'vrijstaande_woning_(waterwoning)',
'vrijstaande_woning_(wind-_of_watermolen)',
'woonboerderij',
'woonboot'}

II. Processed unique house type values

{'2-onder-1-kapwoning',
'bedrijfs-_of_dienstwoning',
'bungalow',
'dijkwoning',
'drive-in_woning',
'eengezinswoning',
'eindwoning',
'geschakelde_woning',
'grachtenpand',
'halfvrijstaande_woning',
'herenhuis',
'hoekwoning',
'hofjeswoning',
'kwadrant_woning',
'landgoed',
'landhuis',
'paalwoning',
'patiowoning',
'semi-bungalow',
'split-level_woning',
'stacaravan',
'tussenwoning',
'verspringend',
'villa',
'vrijstaande_woning',
'waterwoning',
'wind-_of_watermolen',
'woonboerderij',
'woonboot',
'woonwagen'}

III. Unprocessed unique roof type values

{'bitumineuze_dakbedekking',
'dwarskap',
'dwarskap_bedekt_met_asbest',
'dwarskap_bedekt_met_bitumineuze_dakbedekking',
'dwarskap_bedekt_met_kunststof',
'dwarskap_bedekt_met_leisteen',
'dwarskap_bedekt_met_metaal',
'dwarskap_bedekt_met_overig',
'dwarskap_bedekt_met_pannen',
'dwarskap_bedekt_met_riet',
'kunststof',
'leisteen',
'lessenaardak',
'lessenaardak_bedekt_met_asbest',
'lessenaardak_bedekt_met_bitumineuze_dakbedekking',
'lessenaardak_bedekt_met_kunststof',
'lessenaardak_bedekt_met_leisteen',
'lessenaardak_bedekt_met_metaal',
'lessenaardak_bedekt_met_overig',
'lessenaardak_bedekt_met_pannen',
'lessenaardak_bedekt_met_riet',
'mansarde_dak',
'mansarde_dak_bedekt_met_asbest',
'mansarde_dak_bedekt_met_bitumineuze_dakbedekking',
'mansarde_dak_bedekt_met_kunststof',
'mansarde_dak_bedekt_met_leisteen',
'mansarde_dak_bedekt_met_metaal',
'mansarde_dak_bedekt_met_overig',
'mansarde_dak_bedekt_met_pannen',
'mansarde_dak_bedekt_met_riet',
'metaal',
'overig',
'pannen',
'plat_dak',

'plat_dak_bedekt_met_asbest',
'plat_dak_bedekt_met_bitumineuze_dakbedekking',
'plat_dak_bedekt_met_kunststof',
'plat_dak_bedekt_met_leisteen',
'plat_dak_bedekt_met_metaal',
'plat_dak_bedekt_met_overig',
'plat_dak_bedekt_met_pannen',
'riet',
'samengesteld_dak',
'samengesteld_dak_bedekt_met_asbest',
'samengesteld_dak_bedekt_met_bitumineuze_dakbedekking',
'samengesteld_dak_bedekt_met_kunststof',
'samengesteld_dak_bedekt_met_leisteen',
'samengesteld_dak_bedekt_met_metaal',
'samengesteld_dak_bedekt_met_overig',
'samengesteld_dak_bedekt_met_pannen',
'samengesteld_dak_bedekt_met_riet',
'schilddak',
'schilddak_bedekt_met_bitumineuze_dakbedekking',
'schilddak_bedekt_met_kunststof',
'schilddak_bedekt_met_leisteen',
'schilddak_bedekt_met_metaal',
'schilddak_bedekt_met_overig',
'schilddak_bedekt_met_pannen',
'schilddak_bedekt_met_riet',
'tentdak',
'tentdak_bedekt_met_asbest',
'tentdak_bedekt_met_bitumineuze_dakbedekking',
'tentdak_bedekt_met_kunststof',
'tentdak_bedekt_met_leisteen',
'tentdak_bedekt_met_metaal',
'tentdak_bedekt_met_overig',
'tentdak_bedekt_met_pannen',
'tentdak_bedekt_met_riet',
'unspecified',
'zadeldak',
'zadeldak_bedekt_met_asbest',
'zadeldak_bedekt_met_bitumineuze_dakbedekking',
'zadeldak_bedekt_met_kunststof',
'zadeldak_bedekt_met_leisteen',
'zadeldak_bedekt_met_metaal',
'zadeldak_bedekt_met_overig',
'zadeldak_bedekt_met_pannen',
'zadeldak_bedekt_met_riet'}

IV. Processed roof type values

{'',
'asbest',
'bitumineuze_dakbedekking',
'dwarskap',
'kunststof',
'leisteen',
'lessenaardak',
'mansarde_dak',
'metaal',
'overig',
'pannen',
'plat_dak',
'riet',
'samengesteld_dak',
'schilddak',
'tentdak',
'zadeldak'}

D. Final generated model pipeline

```
# pipeline_gen_20_idx_1_2022.06.16_11-06-39.py
import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.linear_model import LassoLarsCV
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline, make_union
from tpot.builtins import StackingEstimator
from tpot.export_utils import set_param_recursive

# NOTE: Make sure that the outcome column is labeled 'target' in the data
file
tpot_data = pd.read_parquet("output/houses_encoded_place_skl.par-
quet").rename(columns={"days_to_sell": "target"})
features = tpot_data.drop('target', axis=1)
training_features, testing_features, training_target, testing_target = \
      train_test_split(features, tpot_data['target'], random_state=42)

# Average CV score on the training set was: -596.6426262034181
exported_pipeline = make_pipeline(
  StackingEstimator(estimator=LassoLarsCV(normalize=False)),
  GradientBoostingRegressor(alpha=0.9, learning_rate=0.1, loss="ls",
max_depth=10, max_features=0.05, min_samples_leaf=5,
min_samples_split=15, n_estimators=100, subsample=0.9000000000000001)
)
# Fix random state for all the steps in exported pipeline
set_param_recursive(exported_pipeline.steps, 'random_state', 42)

exported_pipeline.fit(training_features, training_target)
results = exported_pipeline.predict(testing_features)
```